



Concurrent Programming

Week 10 (Lecture 3)

Stelios Tsampas

February 17, 2025

stelios@imada.sdu.dk

<https://www.steliotsampas.com>

stelios-tau/cp-2025

cp-2025

Previously, on Week 8

Picture a chef (or more than one) cooking multiple dishes at the same time:

The chef prepares multiple meals at once by switching tasks.

Picture a chef (or more than one) cooking multiple dishes at the same time:

The chef prepares multiple meals at once by switching tasks.

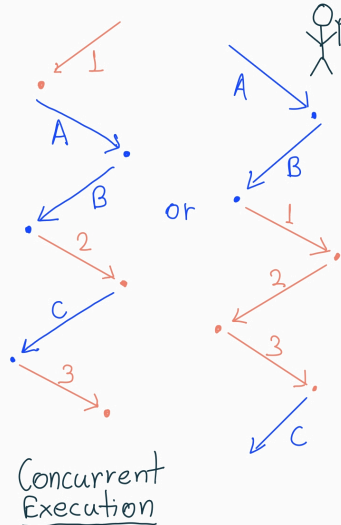
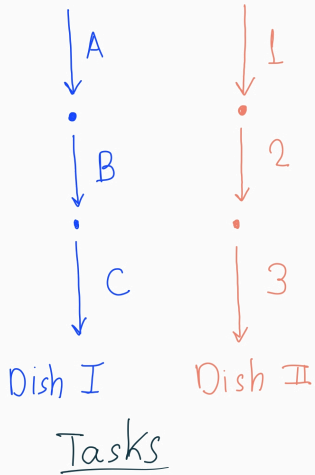
They might chop vegetables, then stir a sauce, then check the oven.

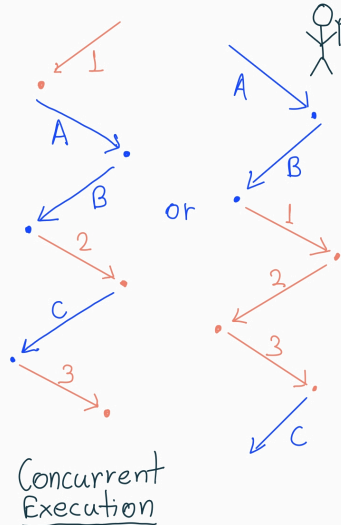
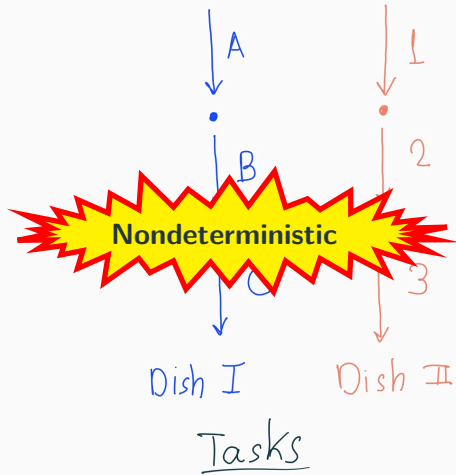
Picture a chef (or more than one) cooking multiple dishes at the same time:

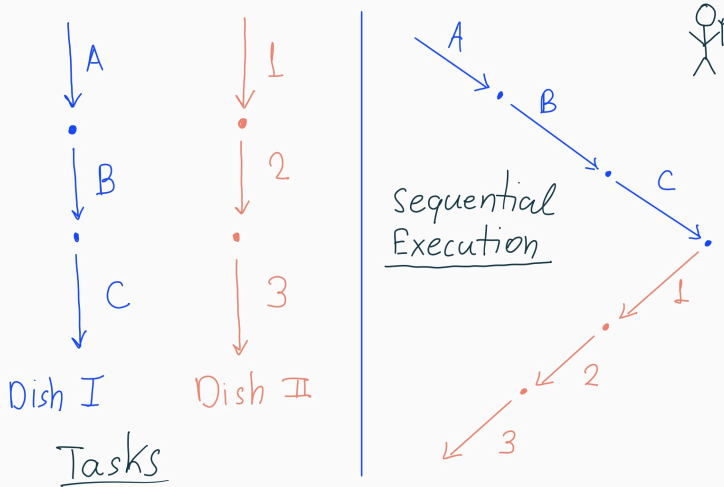
- The chef prepares multiple meals at once by switching tasks.

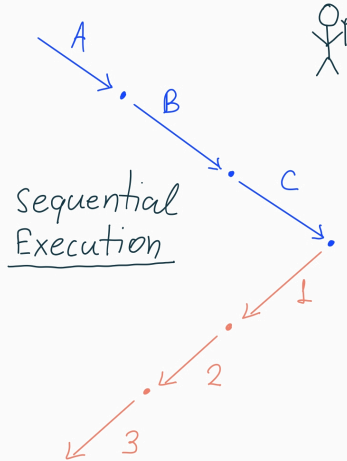
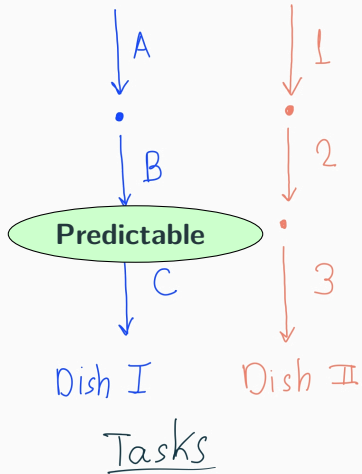
- They might chop vegetables, then stir a sauce, then check the oven.

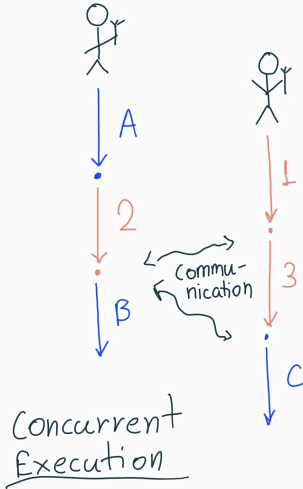
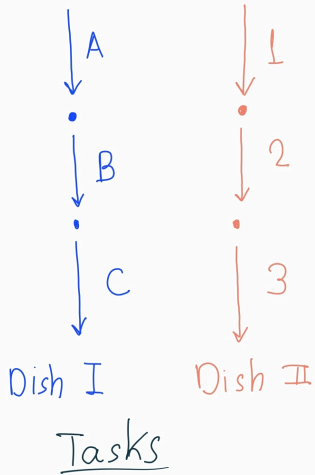
- The dishes are interleaved but not truly parallel.

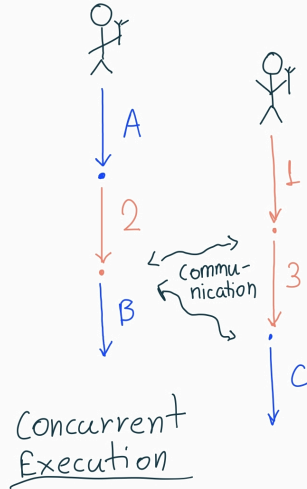
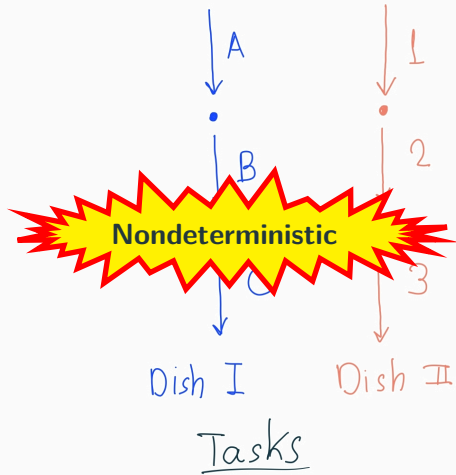


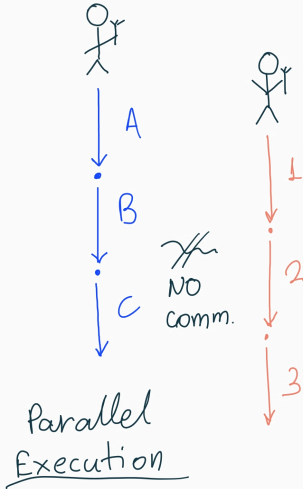
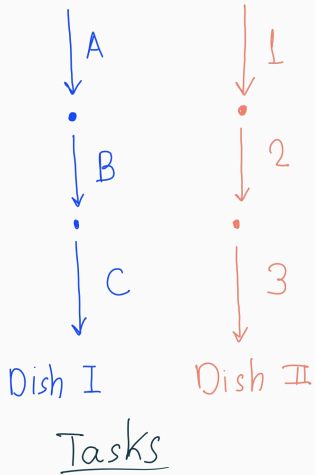


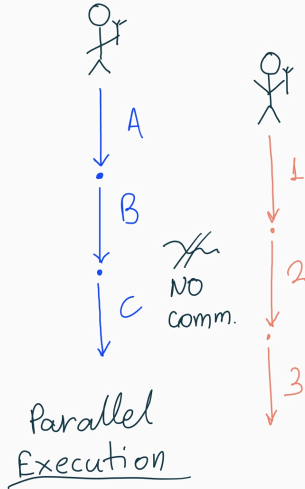
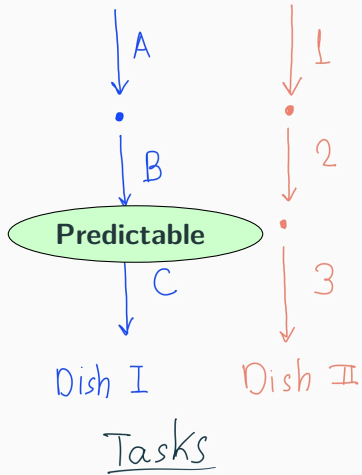












A large amount of this class is about managing the **sharing** of resources and generally the **communication** and **coordination** of these **threads** of execution.

A large amount of this class is about managing the **sharing** of resources and generally the **communication** and **coordination** of these **threads** of execution. Yes, the term *threads* is the technically correct term.

The main abstraction that is involved in concurrency.

A thread represents a lightweight, independent unit of execution.

Threads are present at multiple levels of the software stack:

- Operating system level, i.e. user and kernel threads.

- At the level of programming languages, Java threads, pthreads in C...

There can be many active threads active at the same time, i.e. *concurrently*.

The main abstraction that is involved in concurrency.

A thread represents a lightweight, independent unit of execution.

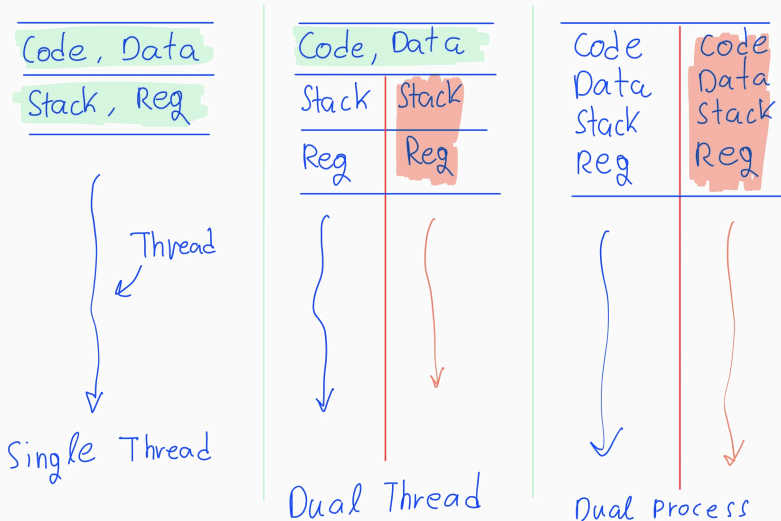
Threads are present at multiple levels of the software stack:

- Operating system level, i.e. user and kernel threads.

- At the level of programming languages, Java threads, pthreads in C...

There can be many active threads active at the same time, i.e. *concurrently*.

... but they do not need to run at the actual same moment.



Atomicity and race conditions

Suppose that two steps of the recipe for [Dish I](#) read as

1. Put veggies in the (heated) frying pan. Wait 8 minutes. Take veggies out.
2. Put meat in the (heated) frying pan. Wait 8 minutes. Take meat out.

Question 1

Question 2

Suppose that two steps of the recipe for [Dish I](#) read as

1. Put veggies in the (heated) frying pan. Wait 8 minutes. Take veggies out.
2. Put meat in the (heated) frying pan. Wait 8 minutes. Take meat out.

Question 1

Is everything alright with this recipe?

Question 2

Suppose that two steps of the recipe for [Dish 1](#) read as

1. Put veggies in the (heated) frying pan. Wait 8 minutes. Take veggies out.
2. Put meat in the (heated) frying pan. Wait 8 minutes. Take meat out.

Question 1

Is everything alright with this recipe?

Question 2

What if two chefs (i.e. two threads) are working on the recipe?

Thread-safe class

A class (or piece of code) is *thread-safe* if it behaves correctly when accessed from multiple threads, regardless of the scheduling or interleaving of the execution of those threads by the runtime environment, and with no additional synchronization or other coordination on the part of the calling code.

Example

Thread-safe class

A class (or piece of code) is *thread-safe* if it behaves correctly when accessed from multiple threads, regardless of the scheduling or interleaving of the execution of those threads by the runtime environment, and with no additional synchronization or other coordination on the part of the calling code.

Example

The *Dish / recipe* is not thread-safe.

Important: There is not pre-existing notion of correctness. The developer decides what constitutes what is correct and what is not.

Race condition

A race condition occurs when the correctness of a computation depends on the relative timing or interleaving of multiple threads by the runtime.

More specific than thread-safety.

Example

Race condition

A race condition occurs when the correctness of a computation depends on the relative timing or interleaving of multiple threads by the runtime.

More specific than thread-safety.

Example

*The correct completion of the recipes by two chefs depends on the timing of the usage of the frying pan. In other words, we have a **single race condition** in the “2-chefs-2-dishes-kitchen” program.*

The sharing of *mutable* state (e.g. the frying pan) may lead to inconsistent, incorrect behaviour.

Proper, careful synchronization.

The sharing of *mutable* state (e.g. the frying pan) may lead to inconsistent, incorrect behaviour.

Proper, careful synchronization.

Immutable (w.r.t the threads) shared state $\implies$ thread-safe code.

The sharing of *mutable* state (e.g. the frying pan) may lead to inconsistent, incorrect behaviour.

Proper, careful synchronization.

Immutable (w.r.t the threads) shared state $\implies$ thread-safe code.

No shared state $\implies$ thread-safe code.

The sharing of *mutable* state (e.g. the frying pan) may lead to inconsistent, incorrect behaviour.

Proper, careful synchronization.

Immutable (w.r.t the threads) shared state $\implies$ thread-safe code.

No shared state $\implies$ thread-safe code.

How do we safely share mutable state?

Atomicity

An operation or sequence of operations on a shared state is atomic if it is *indivisible*, meaning it cannot be interrupted or observed in an incomplete state.

Atomicity

An operation or sequence of operations on a shared state is atomic if it is *indivisible*, meaning it cannot be interrupted or observed in an incomplete state.

Let S be an atomic operation. According to a thread B , a thread A has either finished S or not yet started S .

Our race conditions are results of operations on shared state not being atomic.

Non-atomicity + interleaving $\implies$ problem!

All memory accesses in Java are atomic.

Except long and double! We might get a *data race* with those.

In particular, things like “++” are *not* atomic!

Never assume you can mix atomic operations into a compound, atomic operation.

More synchronization!

All memory accesses in Java are atomic.

Except `long` and `double`! We might get a *data race* with those.

In particular, things like “++” are *not* atomic!

Never assume you can mix atomic operations into a compound, atomic operation.

More synchronization!

Useful details can be found from official [sources](#).

All memory accesses in Java are atomic.

Except long and double! We might get a *data race* with those.

In particular, things like “++” are *not* atomic!

Never assume you can mix atomic operations into a compound, atomic operation.

More synchronization!

Useful details can be found from official [sources](#).

Next up

How can we make non-atomic operations atomic?

Intrinsic locks

This is Java's simplest form of thread synchronization.

This is Java's simplest form of thread synchronization.

An intrinsic lock (or monitor lock) is Java's built-in locking mechanism.

Every Java object has one intrinsic lock (also called a monitor).

One can use an object's intrinsic lock to *protect* access to a block of code.

A thread **must acquire** the object's lock before executing synchronized code.

When the thread exits the block, the lock is automatically released.


```
1 //The method's instance is the underlying monitor of the lock
2 public synchronized void increment() {
3     count++;
4 }
```

```
5 //You can lock specific blocks. Observe the explicit (this).
6 public void increment() {
7     synchronized(this) {
8         count++;
9     }
10 }
```

```
11 //You can lock static methods to restrict ANY invocation.  
12 public static synchronized void log(String message) {  
13     System.out.println(message);  
14 }
```

Locking and atomicity

If access to some shared state is controlled by a **single** lock, then all operations on it are perceived as atomic: a thread can't access said state while another thread is in the middle of an operation on it.

Important

A shared state may consist of multiple variables. Pay attention to their *dependence*: Synchronizing individual variables separately does not ensure atomicity when variables are related.

Locking and atomicity

If access to some shared state is controlled by a **single** lock, then all operations on it are perceived as atomic: a thread can't access said state while another thread is in the middle of an operation on it.

Important

A shared state may consist of multiple variables. Pay attention to their *dependence*: Synchronizing individual variables separately does not ensure atomicity when variables are related.

Make sure you use only **one** lock per “dependent group” of variables, for operations that must happen as a unit.

Locks “serialize” access to shared resource, thus making operations *sequential*.

This means that there is performance overhead associated to locks.

Make sure you use locks efficiently and protect the minimum amount of code.

CookingBenignRace: “Two-chefs” but correctness is independent of the interleaving.

CookingRace: “Two-chefs” with a shared pan. Race condition is present.

CookingFixed: “Two-chefs” with a shared pan, synchronized.

WithdrawFixed: Synchronized check-then-withdraw example.

Counter.java: Example from Week 8 on a shared counter.

CounterPlus.java: Example from Week 8 on a shared counter.

GasTankRace: Gas storage example with a nice race condition.

GasTankRace: Gas storage example with a nice race condition.

Thank you!